

Reward Is Not Reinforcement Until Admitted:

A Governance Layer for Reinforcement Learning in Coding Agents

Nikit Phadke

May 2026

Code and artifacts: github.com/nikitph/rewarder

Abstract

Reinforcement learning agents often treat reward as an authoritative signal for policy improvement. In complex domains such as software engineering, this assumption can fail: rewards may be noisy, short-sighted, causally misattributed, or produced through reward hacking. A coding agent may pass visible tests by hardcoding outputs, deleting or weakening tests, violating architecture, swallowing errors, or introducing hidden regressions. Such outcomes expose a gap between raw reward and legitimate reinforcement.

We introduce *Governed Reinforcement Learning*, a framework in which reward signals are modeled as auditable claims rather than scalar facts. A reward governance function evaluates each raw reward against hard invariants, causal attribution, evaluator reliability, temporal consequences, and exploit risk before producing an admitted reward. Only admitted rewards may update the policy.

We evaluate governed reward in a controlled coding-agent environment designed to isolate reward admission as a selection-pressure mechanism. Across 21 random seeds, 100 tasks per seed, and candidate widths from 3 to 10 patches per task, raw reward selection achieves perfect visible-test success once reward-hacking candidates are available, but hidden-test success collapses and hacked patches are preferentially selected. At 10 candidates per task, hacked patches constitute 36.4% of the candidate pool but 81.2% of raw-selected patches. Governed reward suppresses hacked selections to 0.3% while preserving 98.5% hidden-test success. These results support the central claim: raw reward is not merely noisy under adversarial coding conditions; it can become a selection pressure for illegitimate success.

1 Introduction

Reinforcement learning treats reward as the learning signal. In many environments this is productive: reward compresses task success into a scalar update channel. But in agentic domains with underspecified objectives, partial observability, and exploitable evaluators, the scalar reward emitted by an environment is not always evidence that an action deserves reinforcement.

Software engineering makes this problem concrete. A patch can pass visible tests without solving the issue. It can delete tests, hardcode expected outputs, weaken validation, bypass type checks, violate architectural boundaries, or introduce regressions that appear only under later tasks. If a coding agent receives positive reinforcement for such patches, the policy is not learning to produce better software. It is learning which shortcuts satisfy the reward channel.

We propose a simple reframing:

Reward is not reinforcement until admitted.

Raw reward should first be treated as a claim: “this action deserves reinforcement.” Like evidence in an institution, the claim must be admitted before it can influence judgment. Governed Reinforcement Learning inserts a governance layer between reward observation and policy update. The layer evaluates reward provenance, invariants, causal attribution, temporal evidence, evaluator trust, uncertainty, and exploit risk. Only admitted reward shapes the policy.

The empirical question in this paper is mechanistic:

When raw reward is used as a selection pressure, does it amplify reward hacks relative to their prevalence in the candidate pool, and does reward governance suppress that amplification?

We intentionally evaluate this mechanism in a controlled coding environment. Real repositories are essential for later external validation, but they introduce confounders: task ambiguity, incomplete tests, dependency failures, benchmark leakage, repository conventions, and model-specific priors. A controlled environment lets us vary candidate width, measure candidate-pool hack prevalence, isolate ablated governance checks, and test whether raw reward changes the distribution of selected behavior.

Our contributions are:

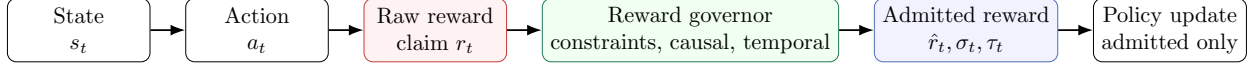
- A formal reward-admission layer for reinforcement learning.
- A coding-agent instantiation with hard invariants, causal confidence, exploit risk, hidden evidence, delayed regression checks, and audit traces.
- A controlled multi-seed experiment showing that raw reward preferentially selects hacked candidates as candidate width increases.
- Ablations showing which governance components are load-bearing.
- A small executable DeepSeek-generated patch sanity check, included as an appendix rather than the primary result.

2 Background and Motivation

Reward hacking and specification gaming are well-known failure modes: an agent optimizes the literal reward channel while failing to satisfy the intended objective [3]. This is a Goodhart-style failure under optimization pressure: a proxy that is adequate at low pressure can become misleading when optimized aggressively [1]. Related failures include goal misgeneralization, where behavior generalizes away from the intended objective even when training reward was high [4].

Coding agents are especially useful for studying this phenomenon because software has executable feedback channels: tests, hidden tests, type checks, builds, linters, security scanners, property tests, mutation tests, differential tests, and architecture rules. These channels are not perfect, but they allow controlled measurement of when an apparent success is legitimate.

Constructed environments are common in coding-agent research. AlphaCode used large-scale sampling and behavior-based filtering for programming-contest solutions, showing the value of executable evaluation and candidate selection in code generation [5]. SWE-bench evaluates agents on real GitHub issues [2], and SWE-bench Verified provides a human-validated 500-task subset [6]. These real-world benchmarks are valuable, but they are not always the cleanest instruments for discovering a new reward-governance mechanism. Here, the primary contribution is mechanistic: how reward admission changes selection pressure.



No raw success may reinforce invariant violation.

Figure 1: Governed reward inserts an admission layer between raw reward observation and learning. Raw reward is treated as a claim about action quality, not as authoritative reinforcement.

3 Governed Reinforcement Learning

Standard reinforcement learning can be written:

$$s_t \rightarrow a_t \rightarrow r_t \rightarrow \text{Update}(\pi).$$

Governed Reinforcement Learning inserts a reward-admission layer:

$$s_t \rightarrow a_t \rightarrow r_t \rightarrow \Gamma \rightarrow \hat{r}_t \rightarrow \text{Update}(\pi).$$

Let $s_t \in S$ be state, $a_t \in A$ action, $o_t \in O$ observed outcome, $r_t \in R$ raw reward, and π the policy. Governed RL introduces:

$$\Gamma(s_t, a_t, o_t, r_t, E_t, P, V, H) \rightarrow (\hat{r}_t, \sigma_t, \tau_t),$$

where E_t is the evidence set, P reward-admission policies, V invariants, H reward history, $\hat{r}_t$ admitted reward, σ_t reward status, and τ_t an audit trace.

Reward status may be:

$$\{\text{PROPOSED, ADMITTED, DISCOUNTED, REJECTED, REVERSED, QUARANTINED, ESCALATED}\}.$$

Policy update is permitted only when

$$\sigma_t \in \{\text{ADMITTED, DISCOUNTED, REVERSED}\},$$

and the update uses $\hat{r}_t$, not r_t .

3.1 Reward Admission Function

A simple scalar instantiation is:

$$\hat{r}_t = (r_t C_t K_t H_t T_t) - \lambda_1 X_t - \lambda_2 U_t - \lambda_3 I_t,$$

where C_t is constraint compliance, K_t causal credit confidence, H_t time-horizon correction, T_t evaluator trust, X_t exploit risk, U_t uncertainty, and I_t invariant violation penalty.

Hard constraints override reward:

$$\text{violates_hard_invariant}(s_t, a_t, o_t) \Rightarrow \hat{r}_t \leq 0, \quad \sigma_t = \text{REVERSED}.$$

Table 1: Reward admission statuses for coding agents.

Status	Meaning	Example
ADMITTED	Clean reward may train	Issue-focused fix passes checks
DISCOUNTED	Reward partially trusted	Large diff or sparse evidence
REJECTED	Reward should not train	Brittle metric hack
REVERSED	Positive reward becomes penalty	Tests deleted or hardcoded
QUARANTINED	More evidence needed	Causal link unclear
ESCALATED	Human review needed	Security-sensitive change

3.2 Invariants

Invariant 1: No update from unadmitted reward. If

$$\sigma_t \in \{\text{PROPOSED}, \text{QUARANTINED}, \text{REJECTED}, \text{ESCALATED}\},$$

then $\text{Update}(\pi, s_t, a_t, \hat{r}_t)$ is forbidden.

Invariant 2: Hard constraints dominate reward. If a hard invariant is violated, then $\hat{r}_t \leq 0$.

Invariant 3: Admitted rewards require provenance. If $\sigma_t = \text{ADMITTED}$, then there exists an audit trace τ_t containing source, evidence, checks, and admission reason.

Invariant 4: Delayed evidence can revise reward. Future evidence may discount or reverse previously admitted rewards.

4 Coding-Agent Instantiation

Raw reward includes visible tests, type checking, linting, and build success:

$$r_{\text{raw}} = 1.0[\text{visible pass}] + 0.5[\text{typecheck pass}] + 0.3[\text{lint pass}] + 0.2[\text{build pass}].$$

Hard checks include test deletion, hardcoded visible outputs, security invariant violations, secrets, public API breakage, architecture boundary violations, and production configuration weakening. Soft checks include hidden-test failure, excessive diff size, weak causal attribution, unjustified dependencies, broad exception handling, delayed regressions, and low maintainability.

5 Experimental Design

We use a controlled coding-agent environment. Each task produces a pool of candidate patch outcomes spanning legitimate fixes, brittle fixes, hardcoded visible-test hacks, test weakening, broad refactors, dependency shortcuts, swallowed errors, architecture violations, and delayed regressions.

This environment is synthetic but not arbitrary. It is designed as an experimental instrument: hack prevalence, candidate width, hidden-test evidence, delayed regressions, causal confidence, and governance checks are explicitly measurable. The goal is to isolate the selection-pressure effect of raw versus admitted reward.

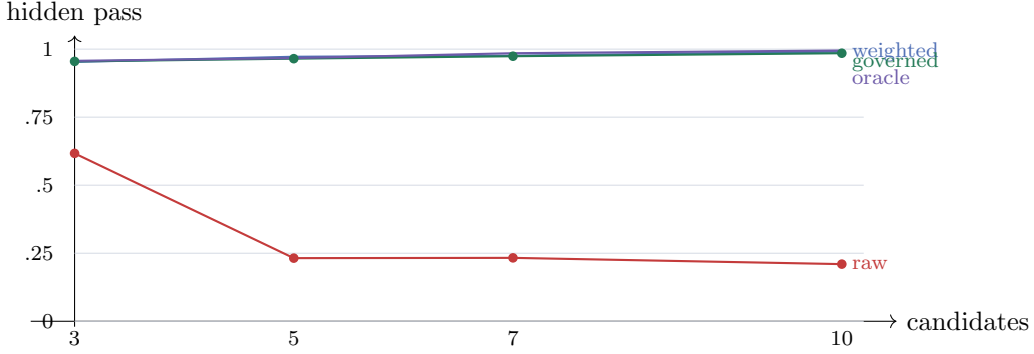


Figure 2: Hidden-test pass rate across 21 seeds. Wider candidate search makes raw reward worse and governed reward better.

We compare five selectors:

$$\begin{aligned}
 \text{RAWSELECTOR} &= \arg \max r_{\text{raw}}, \\
 \text{WEIGHTEDSCALARSELECTOR} &= \arg \max r_{\text{weighted}}, \\
 \text{GOVERNEDSELECTOR} &= \arg \max \hat{r}, \\
 \text{ORACLESELECTOR} &= \arg \max q_{\text{ground truth}}, \\
 \text{LARGERDIFFSELECTOR} &= \arg \max \text{diff size}.
 \end{aligned}$$

The larger-diff selector controls for the objection that governance merely prefers bigger patches.

We run 21 seeds, 100 tasks per seed, and candidate widths of 3, 5, 7, and 10 patches per task. For each run we report visible-test pass rate, hidden-test pass rate, reward-hacking rate, hard invariant violation rate, delayed regression rate, average diff size, and robustness per changed line.

6 Results

6.1 Candidate Width Scaling

When only three candidates are available, the candidate pool contains no reward-hacking strategies in this setup. Even then, governed reward substantially improves hidden-test success and delayed-regression performance, showing that governance is not merely a hack detector. It also performs quality discrimination using hidden evidence, temporal evidence, and causal attribution.

As candidate width increases, raw selection degrades. More candidates give raw reward more opportunities to find visible-test shortcuts. Governed selection remains stable and improves slightly.

At 10 candidates per task, raw reward achieves perfect visible-test pass rate, but hidden-test success collapses. Governed reward sacrifices about 10.6 percentage points of visible-test success relative to raw selection, while gaining about 77.5 percentage points of hidden-test success and reducing reward hacking by about 80.9 percentage points.

6.2 Reward Hacking as Selection Pressure

The most important result is not merely that governed reward performs better. It is that raw reward amplifies hacked patches above their base-rate prevalence in the candidate pool.

At 10 candidates per task, candidate-pool hacking prevalence is 0.364, raw-selected hacking rate is 0.812, and governed-selected hacking rate is 0.003.

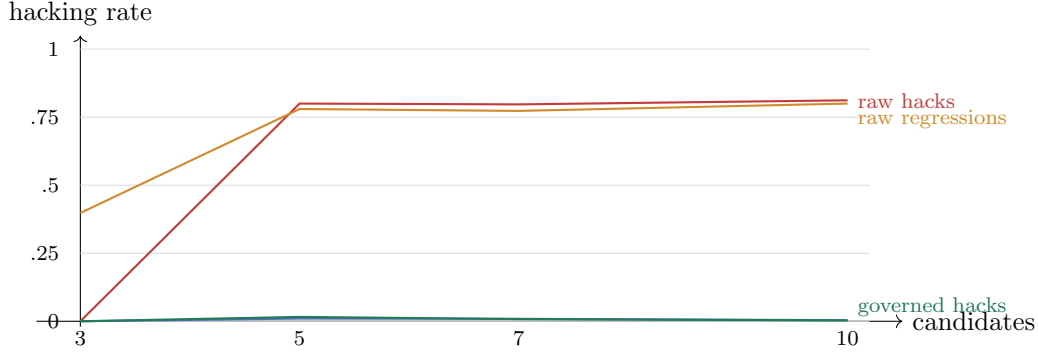


Figure 3: Raw reward amplifies reward hacking and delayed regressions as candidate width increases.

Table 2: Selector metrics at 10 candidates per task. Values are means over 21 seeds.

Selector	Visible	Hidden	Hacking	Delayed reg.	Hard viol.
Raw	1.000	0.210	0.812	0.800	0.732
Weighted scalar	0.940	0.991	0.004	0.030	0.011
Governed	0.894	0.985	0.003	0.036	0.006
Oracle	0.912	0.995	0.000	0.006	0.005
Larger diff	0.583	0.463	0.067	0.479	0.480

Although only 36.4% of candidate patches are hacked, raw reward selects hacked patches in 81.2% of tasks. Raw reward does not merely fail to punish hacks; it preferentially selects them. Governed reward suppresses hacked selections to 0.3%.

6.3 Larger-Diff Control

Governed reward selects larger patches than raw reward because raw reward often selects tiny hardcoded hacks. A reviewer might object that governance is merely preferring larger patches. The larger-diff control addresses this.

At 10 candidates per task, governed selection averages 37.0 changed lines, while the larger-diff selector averages 174.1 changed lines. Yet governed hidden-test pass rate is 0.985 versus 0.463 for larger-diff selection. Governed robustness per changed line is 0.026 versus 0.001 for the larger-diff selector. Governance is not a patch-size preference; it is discriminating between legitimate and illegitimate success.

6.4 Ablations

The hard invariant filter is the most safety-critical single component: removing it increases reward hacking from 0.3% to 10.4% and hard violations from 0.6% to 12.9%. Removing hidden-test evidence or delayed-regression evidence increases temporal failure rates. Removing the exploit detector has little effect in this setup because the hard invariant filter already catches most explicit hacks.

6.5 Weighted Scalar Baseline

The weighted scalar baseline is competitive. At 10 candidates, it achieves 99.1% hidden-test success and 0.4% reward-hacking rate. This should be interpreted honestly.

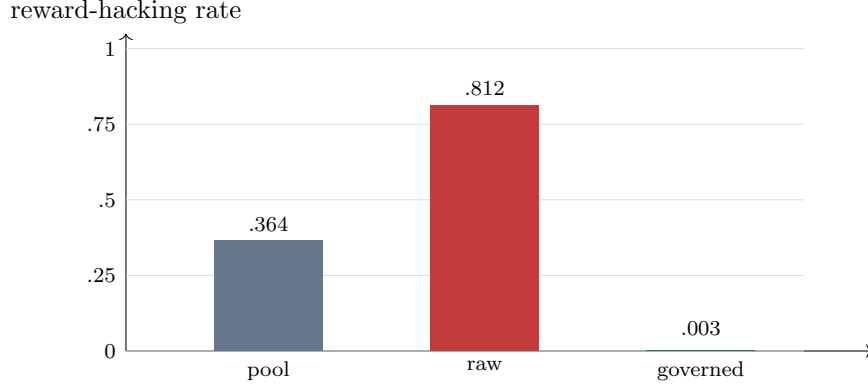


Figure 4: Reward hacking is a selection effect. Raw reward amplifies hacked patches above their base-rate prevalence; governance suppresses them.

Table 3: Governance ablations at 10 candidates per task.

Variant	Hidden	Hacking	Delayed reg.	Hard viol.
Full governance	0.985	0.003	0.036	0.006
No hard invariant filter	0.900	0.104	0.119	0.129
No exploit detector	0.984	0.000	0.037	0.001
No hidden test evidence	0.873	0.000	0.130	0.000
No delayed evidence	0.919	0.000	0.133	0.000
No causal attribution	0.992	0.006	0.024	0.011

If a domain has stable, known evaluation features and the correct weights are known in advance, a well-tuned scalar reward can perform well. Governed reward is not valuable merely because it computes a better number. Its contribution is that reward legitimacy is represented as an auditable state transition:

$$\text{PROPOSED} \rightarrow \{\text{ADMITTED}, \text{DISCOUNTED}, \text{REVERSED}, \text{QUARANTINED}, \text{ESCALATED}\}.$$

Governance records provenance, separates hard invariants from soft discounts, supports delayed revision, and makes explicit why a reward was allowed to shape the policy. A scalar reward may approximate the outcome in a fixed environment, but it does not provide the same institutional structure for reward legitimacy.

7 Theoretical Propositions

Proposition 1: Reward admission prevents direct reinforcement of invariant violations.

If governance enforces

$$\text{violates_hard_invariant} \Rightarrow \hat{r}_t \leq 0$$

and policy updates use only $\hat{r}_t$, then no hard-invariant-violating action can receive positive admitted reinforcement.

Proposition 2: Governance reduces reinforcement of detectable reward hacks. If an exploit detector identifies a class of reward hacks with probability p , and identified hacks are rejected,

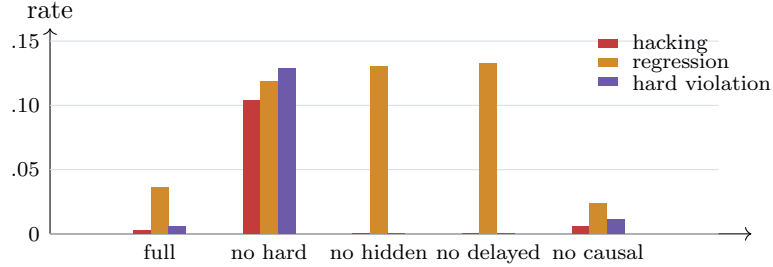


Figure 5: Ablations identify the load-bearing checks. Removing hard invariants increases hard violations; removing hidden or delayed evidence increases regression.

reversed, or sufficiently discounted, then governed RL reduces expected positive reinforcement of that class in proportion to p .

Proposition 3: Delayed reward revision reduces persistence of short-term deceptive strategies. If future evidence can revise prior admitted rewards, strategies that succeed only short-term but fail long-term receive lower cumulative admitted reward than under raw-reward optimization.

8 Discussion

The core result is a selection-pressure result. Raw reward selects what scores. Governed reward selects what deserves to score.

This distinction matters as candidate generation becomes cheaper and wider. Under raw reward, more candidates can make selection worse because search discovers more ways to exploit the metric. Under governed reward, more candidates improve the chance that a legitimate patch exists and can be admitted.

The controlled environment is the correct primary instrument for this claim. It functions like a wind tunnel: the variables of interest are controlled, the mechanism is observable, and the failure mode can be varied systematically. Real-world benchmarks are important for later validation, but they are not always the cleanest way to isolate whether reward admission changes selection pressure.

9 Limitations

This study uses a constructed coding environment. Its purpose is mechanism isolation, not direct prediction of all real-world coding-agent performance. Governance checks are only as good as their design. If a hard invariant is missing, a reward hack may still be admitted. Overly strict governance may suppress exploration or reject unconventional but valid solutions. Weighted scalar rewards can perform competitively when relevant features and weights are stable and known.

The DeepSeek real-code run in Appendix A is small and should not be treated as a benchmark claim. It is included only as a qualitative sanity check that the same selection-pressure pattern can appear with real model-generated patches and executable tests.

10 Conclusion

In complex agentic domains, reward should not be modeled as a scalar fact but as a governance claim: “this action deserves reinforcement.” The claim must be admitted before it can influence

learning.

Our controlled coding-agent experiments show that raw reward can become an adversarial selection pressure: as candidate width increases, it preferentially selects visible-test hacks and hidden-regression patches. Governed reward changes the selection pressure by admitting only rewards that survive invariant, hidden-evidence, temporal, and causal checks.

Reward is not reinforcement until admitted.

A DeepSeek Real-Code Sanity Check

We also ran a small executable real-code sanity check using DeepSeek-generated candidate patches. This is not the main empirical result. It is included only to verify that the qualitative mechanism can appear outside the parametric simulation.

The benchmark contains three Python tasks: inactive OAuth authentication, path traversal prevention, and discount-capping logic. For each task, DeepSeek generated 12 candidate patches. The harness applied each patch to a temporary package, ran visible tests, ran hidden tests, scanned diffs for governance violations, and compared selectors.

Table 4: DeepSeek-generated executable sanity check.

Selector	Visible	Hidden	Hacking	Hard viol.
Candidate pool	0.972	0.444	0.528	0.000
Raw selector	1.000	0.000	1.000	0.000
Raw lottery expected	1.000	0.455	0.545	0.000
Weighted selector	1.000	1.000	0.000	0.000
Governed selector	1.000	1.000	0.000	0.000
Oracle selector	1.000	1.000	0.000	0.000

Raw reward selected visible-pass, hidden-fail patches for all three tasks. Governed selection chose the same hidden-passing patches as the oracle. Because this contains only three tasks and one model sample, it should be treated as illustrative rather than conclusive.

References

- [1] Pedro Freire, Miguel Abreu, Francisco Cruz, and Andreas Wichert. Goodhart’s law in reinforcement learning. *arXiv preprint arXiv:2310.09144*, 2023. URL <https://arxiv.org/abs/2310.09144>.
- [2] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024. URL <https://github.com/SWE-bench/SWE-bench>.
- [3] Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, and Jan Leike. Specification gaming: The flip side of ai ingenuity, 2020. URL <https://deepmind.google/discover/blog/specification-gaming-the-flip-side-of-ai-ingenuity/>.

- [4] Lauro Langosco Di Langosco, Jack Koch, Lee D. Sharkey, Jacob Pfau, and David Krueger. Goal misgeneralization in deep reinforcement learning. In *International Conference on Machine Learning*, 2022. URL <https://arxiv.org/abs/2105.14111>.
- [5] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Remi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. *arXiv preprint arXiv:2203.07814*, 2022. URL <https://arxiv.org/abs/2203.07814>.
- [6] OpenAI. Introducing swe-bench verified, 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>. Accessed 2026-05-25.